ООО «Проектные системы»

УТВЕРЖДЕНАПС.АЛЬФА.001.ПС.2-ЛУ

АВТОМАТИЗИРОВАННАЯ СИСТЕМА «АЛЬФА»

версия 1.0.0

АС «АЛЬФА». REST API. РУКОВОДСТВО РАЗРАБОТЧИКА

АПС.АЛЬФА.001.И4.2

Листов: 11

2026

АС «АЛЬФА» версии 1.0.0 – вымышленная веб-система регистрации и обработки внутренних обращений. Заказчик – условное АО «Северный контур», Исполнитель – условное ООО «Проектные системы». Данные, адреса и процедуры являются проектными примерами; наличие работающего программного продукта и факт испытаний не утверждаются.

Подп. и дата | Инв. № дубл. | Взам. инв. № | Подп. и дата | Инв. № подл.

Копировал Формат А4

Сведения об авторе

Автор демонстрационного документа – Александр Сибиряков, аналитик и технический писатель.

Документ подготовлен специально для профессионального портфолио и демонстрации навыков анализа требований, проектирования и документирования информационных систем. АС «Альфа», организации, данные и проектные ситуации являются условными.

Профессиональный профиль, другие образцы работ и контакты для предложений о сотрудничестве и трудоустройстве:

https://alsibir.github.io/alsibir-expert/

Аннотация

Документ определяет условия вызова методов **REST API**, параметры, ответы, ошибки и конкурентные изменения. Относится к проектной АС «Альфа» версии 1.0.0. Все примеры синтетические; фактические испытания не проводились.

Содержание

1 Назначение API....4
2 Базовые адреса и версии....4
3 Аутентификация и авторизация....4
4 Общие правила формирования запросов....4
5 Общие правила формирования ответов....5
6 Модель ошибок....5
7 Методы REST API и права доступа....5
8 Методы работы с комментариями и вложениями....8
9 Административные методы....8
10 Ограничения, пагинация и фильтрация....8
11 Идемпотентность и конкурентные изменения....8
12 Пример последовательности создания заявки....9
13 Версионирование и совместимость API....10

1 Назначение API

REST API предназначен для UI и разрешённых интеграций. Все изменения проверяются сервером по SEC-001.

2 Базовые адреса и версии

Базовый URL https://alpha.example.test/api/v1. Адрес относится к зарезервированному тестовому домену; это пример конфигурации.

3 Аутентификация и авторизация

Пароль содержит не менее 12 символов и хранится как Argon2id-хеш с солью. После 5 ошибок за 15 мин вход блокируется на 15 мин. Непрозрачный сеанс передаётся браузером в cookie Secure, HttpOnly, SameSite = Strict; простой – 30 мин, абсолютный срок – 8 ч. Для изменяющих запросов браузера проверяется X-CSRF-Token; POST /auth/logout аннулирует сеанс. Интеграции используют отдельные сервисные записи и Bearer-токены со сроком 1 ч; токены в журналах не сохраняются.

4 Общие правила формирования запросов

JSON передаётся в UTF-8 с Content-Type: application/json. Идентификатор заявки в URI – UUID; внешний номер является полем number. Вход принимает login/password.

Изменяющий браузерный запрос передаёт cookie сеанса и X-CSRF-Token; cookie не заменяет проверку владения объектом. Сервисная интеграция передаёт Authorization: Bearer < токен > и использует заранее выданные права; получение сервисного токена относится к администрированию и не является публичным методом регистрации. UUID и значения справочников проверяются до изменения данных. Запрос без обязательного Idempotency-Key к созданию заявки получает 422. Пустой JSON, неизвестное поле, неверный тип и отсутствующее обязательное поле дают 422 с указанием поля.

5 Общие правила формирования ответов

Успешное создание возвращает 201, Location и объект; чтение и изменение – 200. Удаление означает отмену и возвращает 204. Время – ISO 8601 UTC.

6 Модель ошибок

Ошибка имеет поля code, message, trace_id, details. 401 – нет действующего сеанса; 403 – недостаточно полномочий; 404 – объект отсутствует или скрыт; 409 – переход/версия; 413 – размер; 415 – тип; 422 – поля; 423 – блокировка входа; 503 – зависимость недоступна.

Пример:

```
{"code":"VERSION_CONFLICT","message":"Карточка изменена другим
участником","trace_id":"55555555-5555-4555-8555-
555555555555","details":{"expected_version":2,"current_version":3}}.
```

details не содержит секретов и сведений о недоступных пользователю объектах. *404 применяется к чужой заявке, 403 – к известной операции, запрещённой ролью.* ***При ошибке БД или хранилища операция не подтверждается успешным кодом. Клиент сохраняет введённый текст и trace_id для диагностики.***

7 Методы REST API и права доступа

REST API включает операции управления сеансами, заявками, комментариями, вложениями, уведомлениями и учётными записями, а также получения событий аудита.

Для изменения заявки методом PUT передаётся полный набор редактируемых полей: title, description, priority и version. Изменение автора, номера и категории заявки после её создания не допускается.

Методы, права доступа, входные данные, результаты и ограничения приведены в таблице 1.

Таблица 1 – Контракт REST API и права доступа

Метод и URI	Доступ, условия и входные данные	Успешный результат	Исключения и ограничения
POST /auth/login	Без действующего сеанса. Вход: login, password; запись активна либо временно заблокирована.	200: установлена cookie сеанса, возвращён CSRF-токен и сведения о роли.	Неверные реквизиты – 401; действующая блокировка – 423. Пароль и сеанс не журналируются.
GET /auth/session	Любая активная учётная запись; действующий сеанс.	200: идентификатор записи, роль, срок сеанса и CSRF-токен.	Недействующий или отозванный сеанс – 401.
POST /auth/logout	Любая активная учётная запись; действующий сеанс; CSRF-токен для браузерного запроса.	204 без тела; текущий сеанс отозван.	Повторный запрос с отозванным сеансом – 401; другие сеансы не отзываются этой операцией.
GET /requests	Пользователь – свои заявки; Оператор – все заявки. Параметры: page, size, status, priority, q; фильтры необязательны.	200: items, total, page, size; Пользователь видит свои, Оператор – все.	ADMIN и AUDITOR не получают содержимое заявок; неверный фильтр – 422.
POST /requests	Пользователь (USER). Вход: title, description, category, priority; заголовок Idempotency-Key.	201 и Location; id, number, status = NEW, version = 1.	category: ACCESS/EQUIPMENT/ SOFTWARE/OTHER; priority: NORMAL/HIGH. Ограничения полей – 422.
GET /requests/{id}	Автор заявки либо Оператор (OPERATOR). Вход: UUID заявки.	200: карточка, доступные комментарии, метаданные вложений и история.	Отсутствующий или чужой объект – 404. Пароли и секреты в представление не включаются.
PUT /requests/{id}	Автор заявки; статус NEW. Вход: полный набор title, description, priority, version.	200: полная актуальная карточка с увеличенной version.	Автор, номер и категория не изменяются. Устаревшая версия – 409; иной статус – 409; запрещённая роль – 403.
PATCH /requests/{id}/status	Автор заявки либо Оператор – в соответствии с матрицей переходов. Вход: status, version.	200: карточка с новым статусом, аудит и outbox в одной транзакции.	Недопустимый переход или версия – 409; запрет роли – 403; неизвестный статус – 422.
DELETE /requests/{id}	Автор заявки; статус NEW. Заголовок If-Match с ожидаемой версией.	204: CANCELLED, история сохранена.	Это отмена, а не физическое удаление. Повтор после отмены с актуальной версией – 409: исходный статус уже не NEW.

POST /requests/{id}/ comments	Автор заявки либо Оператор; нетерминальный статус заявки. Вход: text длиной 1–2000 символов.	201: id, text, author_id и created_at.	Пустой текст – 422; терминальный статус – 409. Комментарий после создания не редактируется и не удаляется через v1.
POST /requests/{id}/ attachments	Автор заявки либо Оператор; нетерминальный статус заявки. Вход: multipart/form-data, поле file.	201 после проверки: file_id, filename, media_type, size и sha256.	Более 10 МиБ – 413; запрещённый тип – 415; 11-й файл либо терминальный статус – 409; антивирус недоступен – 503; заражение – 422.
GET /requests/{id}/ attachments/{file_id}	Автор заявки либо Оператор; доступ к заявке; опубликованный файл принадлежит этой заявке.	200: поток файла, Content-Type, Content-Disposition с безопасным именем.	Чужой или неизвестный файл – 404; физически недоступное хранилище – 503.
GET /notifications	Любая активная учётная запись. Параметры: page, size.	200: items текущего адресата с id, event_id, request_id, created_at и read_at.	Чтение списка не изменяет read_at; операция отметки прочтения в v1 не заявлена.
GET /users	Администратор (ADMIN). Параметры: page, size.	200: id, login, email, role и blocked без хеша пароля.	Иная роль – 403. Фильтрация по логину в v1 отдельно не заявлена.
POST /users	Администратор (ADMIN). Вход: login, email, role из установленного списка.	201: id и атрибуты записи без пароля; временный секрет выдаётся отдельным защищённым каналом.	Повтор login – 409; отсутствующие поля или неизвестная роль – 422. Порядок первичной смены пароля описан как проектная процедура документа 09.
PATCH /users/{id}	Администратор (ADMIN). Вход: role и/или blocked; требуется хотя бы одно поле.	200: актуальная роль и блокировка; изменение действует со следующего запроса.	Назначение нескольких ролей – 422; неизвестный id – 404; при блокировании отзываются сеансы и сервисные токены записи.
GET /audit	Аудитор (AUDITOR). Параметры: from, to, action, page, size.	200: события с субъектом, объектом, временем, результатом и trace_id.	Требуется уточнение: исключения и ограничения в исходном описании не определены.

Исключения и ограничения: ADMIN не получает право чтения автоматически. Неизвестное значение action – 422; изменение событий не поддерживается.

8 Методы работы с комментариями и вложениями

Вложения: PDF, PNG, JPEG, TXT; не более 10 МиБ на файл и 10 файлов на заявку. Перед публикацией выполняется проверка типа и антивирусная проверка; недоступность проверки приводит к отклонению загрузки с 503. Заявки и вложения хранятся 3 года после закрытия; аудит – 1 год, технические журналы – 30 суток. Физическое удаление выполняет регламентное задание; публичный DELETE отменяет заявку без удаления истории.

9 Административные методы

POST /users создаёт запись; PATCH /users/{id} меняет роль и blocked. GET /users доступен только Администратору. Пароль и хеш никогда не возвращаются.

10 Ограничения, пагинация и фильтрация

GET /requests?page = 1&size = 20&status = NEW; size от 1 до 100; сортировка created_at DESC, id ASC. Фильтр не расширяет права. Поиск q – по теме, длина до 120 символов.

Параметр priority поддерживает NORMAL или HIGH и сочетается с status и q по условию И. page начинается с 1; при выходе за последнюю страницу возвращаются items = [] и фактический total. total считается после применения прав и фильтров. sort в v1 не принимается: порядок фиксирован created_at DESC, id ASC; одинаковое время не нарушает устойчивость порядка. Для GET /notifications, /users и /audit действуют page = 1 и size = 20 по умолчанию, size 1–100; порядок occurred_at либо created_at DESC, id ASC в соответствии с ресурсом. from/to аудита – ISO 8601 UTC, from включительно, to исключительно; from > = to даёт 422.

11 Идемпотентность и конкурентные изменения

POST /requests требует Idempotency-Key UUID. Ключ хранится 24 ч вместе с хешем тела и ответом; повтор того же тела возвращает исходный результат, другого тела – 409. PUT и смена статуса используют version; устаревшая версия даёт 409.

При повторе POST с тем же ключом, действующей авторизацией и тем же каноническим JSON сервер возвращает сохранённые 201, Location и тело исходного ответа. Другой пользователь не может получить чужой результат по ключу: область ключа включает субъекта и операцию. Иное тело при том же ключе даёт 409. После истечения 24 ч ключ не гарантирует защиту от повторной регистрации; при неопределённом результате сначала проверяется список заявок. Одновременные первые запросы с одним ключом сериализуются, а не создают две заявки.

Пример конфликта: Автор прочитал version = 2, Оператор изменил карточку до version = 3. PUT с version = 2 получает 409 без записи. Клиент перечитывает карточку, показывает актуальные данные и предлагает повторить изменение осознанно; **автоматическая перезапись новой версии запрещена**. DELETE /requests/{id} требует ожидаемую версию в заголовке If-Match: "2"; отсутствие заголовка даёт 422, несовпадение – 409. Это конкретизирует проверку конкурентных изменений при отмене.

12 Пример последовательности создания заявки

Клиент выполняет вход, получает CSRF-токен через GET /auth/session и отправляет POST /requests. Сервер фиксирует заявку, аудит и outbox; пример приведён ниже.

```
POST /api/v1/requests
Idempotency-Key: 44444444-4444-4444-8444-444444444444
{"title":"Не работает принтер","description":"Принтер в комнате 12 не
принимает задания","category":"EQUIPMENT","priority":"NORMAL"}
201 Created
{"id":"11111111-1111-4111-8111-111111111111","number":"ALF-
000001","status":"NEW","version":1}
```

Порядок проверок и граница транзакции представлены на рисунке.

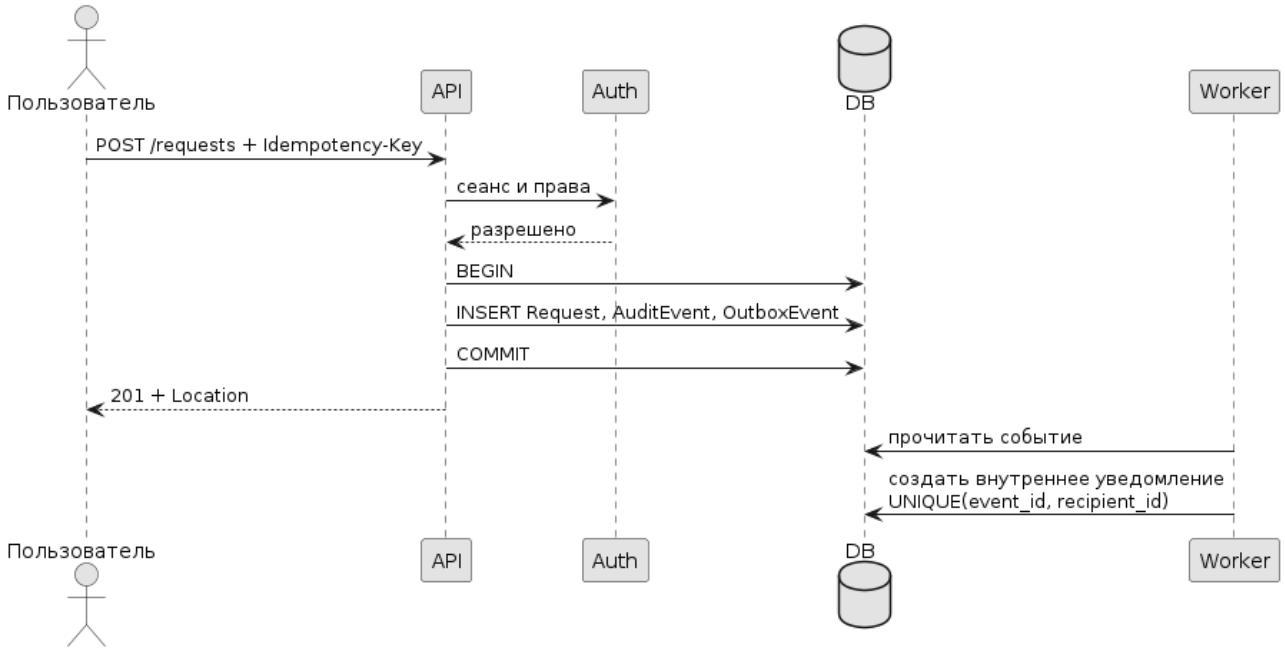


Рисунок 1 – Создание заявки через API

13 Версионирование и совместимость API

Совместимые необязательные поля добавляются в v1; удаление полей или изменение семантики требует v2. Неизвестные поля запроса отклоняются с 422.

Минимальный успешный обмен:

```
GET /requests?page = 1&size = 20&priority = HIGH → 200
{"items":[],"total":0,"page":1,"size":20}. Пример ошибки поля: 422
{"code":"VALIDATION_ERROR","message":"Проверьте поля","trace_id":"55555555-
5555-4555-8555-555555555555","details":{"title":"Длина от 5 до 120
символов"}}.
```

Примеры являются проектными контрактами, а не записанными ответами работающего стенда.

Справочник не изменяет семантику уже сохранённых кодов. Необязательное поле ответа может добавляться в v1, если клиент игнорирует неизвестные поля ответа; строгая проверка неизвестных полей относится только к запросам. Изменение обязательности, типов, смысла статусов либо отмена метода требуют новой основной версии API и согласованного плана перехода.

Лист регистрации изменений

Изм.	Номера листов (страниц)				Всего листов (страниц) в документе	Номер документа	Входящий номер сопроводительного документа и дата	Подпись	Дата
	изме-ненных	заме-ненных	новых	аннули-рованных					